Git سيناريوهات

الأمر	الاستخدام باختصار
squash	دمج عدة كوميتات في كوميت واحد
merge / rebase	دمج الفروع بتاريخ متفرع أو خطي
help	قراءة التوثيق الرسمي
cherry-pick	نقل كوميت محدد بين الفروع
clean	حذف الملفات غير المتتبَّعة
grep	البحث داخل الشيفرة والتاريخ
blame	معرفة صاحب كل سطر وسياقه
bisect	إيجاد الكوميت المسبب للخطأ
shortlog	تلخيص الكوميتات حسب المؤلف
prune	حذف الكائنات اليتيمة
worktree	العمل على عدة فروع في مجلدات منفصلة
verify-commit	التحقق من التوقيع الرقمي للكوميت
filter-repo	إعادة كتابة التاريخ لحذف الملفات أو الأسرار

1. git squash (دمج الكوميتات)

هو أسلوب يُنفَّذ عبر `git rebase -i` أو `git merge --squash`.

السيناريو: أنهيتُ تطوير ميزة تسجيل الدخول، وتراكمت لدي ثمانية كوميتات مثل «تعديل بسيط» و«إصلاح خطأ إملائي» و«تجربة». قبل فتح طلب الدمج (Pull Request) أردتُ تاريخاً نظيفاً يسهل على المراجع قراءته. نفّذتُ `git rebase -i HEAD~8` وحوّلتُ الكلمة `pick` إلى `squash` للكوميتات السبعة الأخيرة، فصارت الميزة كوميتاً واحداً برسالة واضحة. **السبب:** تسهيل المراجعة وإبقاء تاريخ الفرع الرئيسي مرتباً.

2. git rebase مقابل git merge

الفرق: الأمر `rebase` فيعيد تطبيق كوميت دمج ويحافظ على التاريخ الحقيقي المتفرع، أما `merge` ينشئ كوميتاتك فوق آخر نسخة من الفرع الأساسي فينتج تاريخ خطي، لكنه يغيّر معرّفات الكوميتات.

السيناريو: أعمل على فرع خاص بي `feature/search` ومرّ يومان تقدّم خلالهما فرع `main` بعدة كوميتات من زملائي. بما أن الفرع خاص بي ولم يشاركه أحد، نفّذتُ `git rebase main` ليبدو عملي كأنه بدأ من آخر نسخة، وبعدها دمجته بسهولة. أما عند دمج الفرع المشترك فأستخدم `main` في `develop` للفروع الخاصة لتاريخ `rebase` **:السبب** حتى لا أعيد كتابة تاريخ يعتمد عليه فريق كامل. `git merge` نظيف، و `merge` للفروع المشتركة حفاظاً على سلامة التاريخ.

3. git help

السيناريو: أحتاج إلى حفظ تغييراتي مؤقتاً لكني لا أتذكر خيارات `git stash`. نفّذتُ `git help stash` فظهرت الصفحة الكاملة بالخيارات والأمثلة، وعندما أردتُ تذكيراً سريعاً فقط استخدمتُ `git stash -h`. **السبب:** الحصول على توثيق رسمي موثوق `git help -g` كذلك يعرض أدلة المفاهيم مثل دليل الفروع. دون مغادرة الطرفية.

4. git cherry-pick

السيناريو: اكتشف فريق الدعم ثغرة في النسخة المنشورة `release/2.0`، وأصلحها زميلي بكوميت واحد داخل فرع `develop` كله، فنفّذتُ على `develop` الذي يحوي أيضاً ميزات غير مكتملة. لا يمكنني دمج فرع الإصدار `git cherry-pick -x <hash>` فانتقل الإصلاح وحده، وأضاف الخيار `-x` إشارة إلى الكوميت الأصلي في الرسالة. **السبب:** نقل تغيير محدد بين الفروع دون جلب بقية الأعمال.

5. git clean

السيناريو: بعد أسابيع من البناء والتجارب امتلأ مجلد المشروع بملفات مؤقتة وسجلات ومخرجات بناء غير متتبَّعة. قبل التنظيف نفّذتُ `git clean -nd` لمعاينة ما سيُحذف دون حذفه فعلياً، وبعد التأكد أن الملفات المهمة غير مشمولة نفّذتُ `git clean -fd`. **السبب:** إعادة المجلد إلى حالة نظيفة تطابق المستودع. **تنبيه:** الحذف لا يمكن التراجع عنه، لذا تُستخدم المعاينة `-n` دائماً أولاً.

6. git grep

السيناريو: قررنا تغيير اسم دالة `getUserData` وأريد معرفة كل الأماكن التي تستخدمها. نفّذتُ `git grep -n "getUserData"` فحصلتُ فوراً على أسماء الملفات وأرقام الأسطر، وهو أسرع من البحث العادي لأنه يبحث في الملفات المتتبَّعة فقط. وللتحقق من وجودها في إصدار قديم نفّذتُ `git grep "getUserData" v1.0`. **السبب:** بحث سريع ودقيق داخل الشيفرة وعبر نسخ التاريخ.

7. git blame

السيناريو: وجدتُ في ملف `payment.js` سطراً غريباً يتجاوز التحقق من المبلغ، وأريد فهم سبب وجوده. نفّذتُ `git blame -L 40,45 payment.js` فعرف لي من عدّل كل سطر ومتى ورقم الكوميت، ثم فقرأتُ رسالته وفهمتُ أنه حل مؤقت لمشكلة مع بوابة دفع. **السبب:** معرفة `git show` فتحتُ الكوميت بـ سياق التعديل وصاحبه لسؤاله، وليس لتوجيه اللوم.

8. git bisect

السيناريو: كان التطبيق يعمل جيداً في النسخة الموسومة `v1.2` ، وفي الأسبوع الحالي توقف زر الدفع عن العمل بين مئتي كوميت. بدلاً من فحصها كلها نفّذتُ `git bisect start` ثم `git bisect bad` للحالة الحالية و `git bisect good v1.2` يختار Git وبعد نحو الكوميت الأوسط فأختبره وأخبره بالنتيجة، ثماني خطوات فقط حدّد الكوميت المسبب. ويمكن أتمتة العملية بـ `git bisect run ./test.sh` ، **السبب:** الوصول إلى الكوميت المعطِّل بسرعة بالبحث الثنائي. وننهيها بـ `git bisect reset`.

9. git shortlog

السيناريو: عند إصدار النسخة 3.0 طلب مدير المشروع تقرير مساهمات الفريق وملاحظات الإصدار. فحصلتُ على عدد كوميتات كل مطوّر مرتباً نفّذتُ `git shortlog -sn --no-merges v2.0..v3.0` أحصل على قائمة بعناوين كوميتات كل شخص أستخدمها في كتابة ملاحظات `-sn` تنازلياً، وبدون الخيارين الإصدار. **السبب:** تلخيص النشاط حسب المؤلف لإعداد التقارير.

10. git prune

السيناريو: مستودع المشروع تضخم حجمه بعد كثرة إعادة كتابة الكوميتات والفروع المحذوفة، وتراكمت كائنات (objects) لمعاينة الكائنات اليتيمة، ثم `git prune -n -v` لا يشير إليها أي فرع أو وسم. نفّذتُ هذه المهمة تلقائياً. **السبب:** استعادة مساحة التخزين. `git gc` لحذفها. وفي الغالب يتولى `git prune` الذي يزيل مراجع الفروع البعيدة المحذوفة فقط `git remote prune origin` **تنبيه:** يختلف عن.

11. git worktree

السيناريو: أعمل منذ ساعات على ميزة كبيرة بملفات غير جاهزة للحفظ، وفجأة وصل بلاغ عاجل يتطلب إصلاحاً على `main` . بدلاً من `git stash` أو نسخ المشروع نفّذتُ `git worktree add ../hotfix` مجلداً مستقلاً مرتبطاً بنفس المستودع أصلحتُ فيه الخطأ وأرسلتُه، ثم عدتُ إلى عملي Git فأنشأ ، `main` **السبب:** العمل على أكثر من فرع في وقت واحد دون تعطيل العمل الجاري. كما هو. وبعد الانتهاء حذفتُ المجلد بـ `git worktree remove ../hotfix` .

12. git verify-commit

السيناريو: نعمل في مشروع مفتوح المصدر حساس، وقبل دمج كوميت من مساهم خارجي أريد التأكد أنه فأظهر صحة التوقيع `git verify-commit -v <hash>` موقّع فعلاً بمفتاحه ولم يُنتحل اسمه. نفّذتُ وهوية الموقِّع. **السبب:** حماية سلسلة التوريد البرمجية والتأكد من أصالة الكوميت (SSH أو GPG) الرقمي للوسوم بنفس الفكرة `git verify-tag` وعدم العبث به. ويُستخدم.

13. git filter-repo

`git` وهي البديل الموصى به لـ ، `pip install git-filter-repo` **ملاحظة:** هي أداة خارجية تُثبَّت بـ `filter-branch`.

يحتوي على مفاتيح قاعدة البيانات رُفع بالخطأ منذ `.env` **السيناريو:** اكتشف أحد المطورين أن ملف جديدة من المستودع (clone) شهور، وحذفه في كوميت جديد لا يكفي لأنه يبقى في التاريخ. أنشأتُ نسخة فأُزيل الملف من كل كوميتات التاريخ، `git filter-repo --path .env --invert-paths` ونفّذتُ clone، وطلبتُ من الفريق إعادة الـ . `git push --force` ثم أعدتُ إضافة المستودع البعيد ورفعتُه بـ وغيّرنا كلمات المرور المسرّبة فوراً. **السبب:** إزالة البيانات الحساسة أو الملفات الضخمة من التاريخ بالكامل. **تنبيه:** العملية تعيد كتابة التاريخ كله وتغيّر كل معرّفات الكوميتات.

ملخص سريع